

Brain Tumor Classification using Self-Supervised Contrastive Learning (SimCLR)

MIC Course Project Report

Aadeshveer Singh (24B0926)

Ansh Garg (24B1066)

May 3, 2026

Abstract

Brain tumor classification from MRI scans is an important problem in medical image analysis. Labelled medical data is often scarce and expensive to obtain, which makes supervised deep learning difficult. In this project, we apply **SimCLR** — a self-supervised contrastive learning framework — to learn strong visual representations from MRI images without relying on labels during pre-training. We then fine-tune the learned encoder on a 3-class brain tumor dataset and achieve a test accuracy of **94.13%**, demonstrating that self-supervised pre-training can be highly effective even when labelled data is limited.

Contents

1	Introduction	3
2	Dataset	4
2.1	Overview	4
2.2	Image Characteristics	4
2.3	Data Extraction and Preprocessing	5
2.4	Patient-Level Data Splitting	6
3	Method: SimCLR	7
3.1	Why Self-Supervised Learning?	7
3.2	Core Idea	7
3.3	Step-by-Step Walkthrough	8
3.4	Why This Works	10
3.5	Fine-tuning for Classification	10
4	Code Walkthrough	10
4.1	Data Organizer (<code>extract.py</code>)	11
4.2	Main Training Script (<code>main.py</code>)	11
5	Results	12
5.1	Pre-training Loss	12
5.2	Fine-tuning Accuracy	13
5.3	Test Accuracy	14
6	Comparison	14
7	Discussion	16
8	Conclusion	16
	Appendix: Full Training Logs	17

1 Introduction

Brain tumors are one of the most serious neurological conditions, and early, accurate classification is critical for choosing the right treatment. MRI (Magnetic Resonance Imaging) is the standard imaging tool used for this purpose, as it gives detailed soft-tissue contrast without radiation.

Traditionally, radiologists manually inspect MRI slices to classify tumors — a process that is time-consuming, subjective, and prone to fatigue-related errors. Deep learning offers a way to automate this process, but it typically requires large amounts of carefully labelled training data. In medicine, labelling is expensive because it requires expert radiologists.

This project explores whether we can sidestep this bottleneck using **self-supervised learning**. Specifically, we implement SimCLR, a method that teaches a neural network to understand images *without any labels*. The core idea is simple: if you take the same image and apply two different random augmentations, the network should recognize that both versions come from the same image. This forces it to learn meaningful visual features. We then take this pre-trained network and fine-tune it on the labelled tumor data.

Phase 1: Self-Supervised Pre-training (no labels)

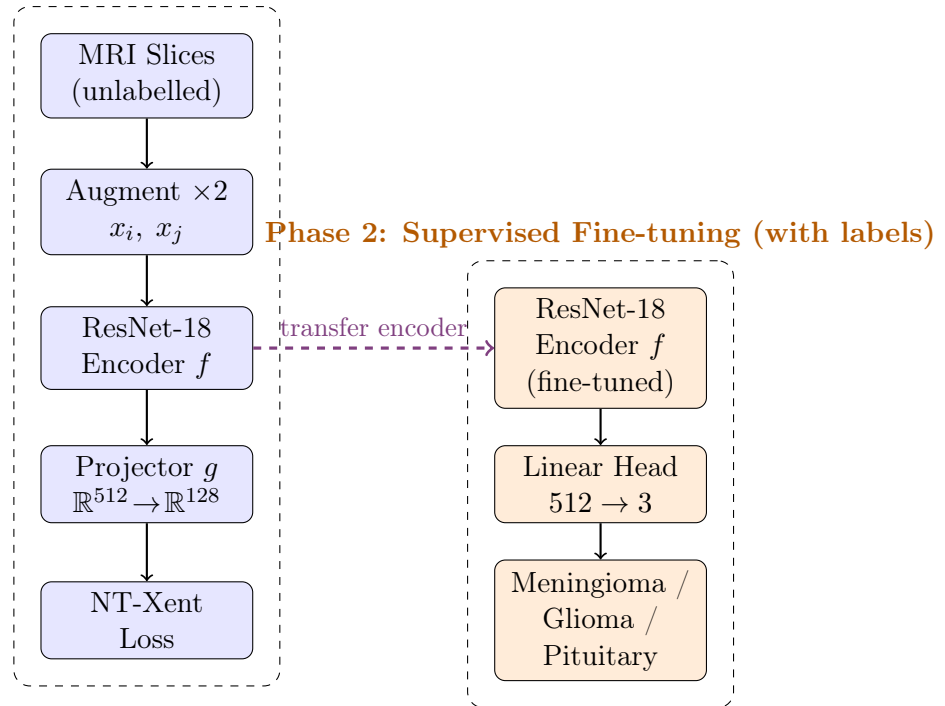


Figure 1: High-level overview of our two-phase pipeline. In Phase 1, the encoder is trained using SimCLR’s contrastive objective on unlabelled MRI slices. In Phase 2, the projector is discarded and a linear classifier is trained on top of the frozen (then fine-tuned) encoder using class labels.

2 Dataset

2.1 Overview

We use the publicly available **Brain Tumor MRI Dataset** created by Jun Cheng at the School of Biomedical Engineering, Southern Medical University, Guangzhou, China. The dataset was originally published to support the following papers:

1. Cheng et al., “Enhanced Performance of Brain Tumor Classification via Tumor Region Augmentation and Partition,” *PLoS ONE*, 2015.
2. Cheng et al., “Retrieval of Brain Tumors by Adaptive Spatial Pooling and Fisher Vector Representation,” *PLoS ONE*, 2016.

The dataset consists of **3,064 T1-weighted contrast-enhanced MRI slices** collected from **233 patients**. The images cover three distinct types of brain tumor:

Tumor Type	Label	Slices	Fraction
Meningioma	1	708	23.1%
Glioma	2	1426	46.6%
Pituitary	3	930	30.4%
Total	—	3064	100%

Table 1: Class distribution in the brain tumor dataset.

The dataset is notably **imbalanced**, with glioma having nearly twice as many samples as meningioma. This is important to keep in mind when interpreting results, as a model could achieve relatively high accuracy by simply favouring the majority class.

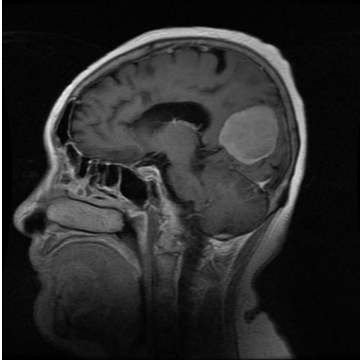
2.2 Image Characteristics

All images are T1-weighted contrast-enhanced MRI scans. In T1-weighted imaging, fat appears bright and water appears dark, which gives good contrast for soft tissues. The “contrast-enhanced” part means a contrast agent (usually Gadolinium) was injected before scanning — this highlights blood vessels and abnormal tissue, making tumors stand out more clearly.

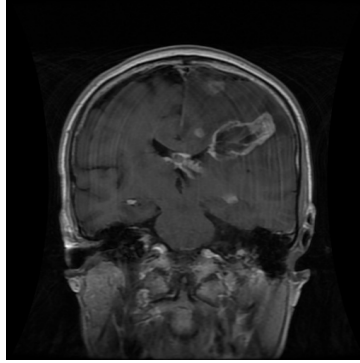
Each `.mat` file in the dataset stores a MATLAB struct (`cjdata`) with the following fields:

- `cjdata.image`: The 2D grayscale MRI slice.
- `cjdata.label`: Tumor type (1 = meningioma, 2 = glioma, 3 = pituitary).
- `cjdata.PID`: Patient ID, a string used to group slices belonging to the same person.

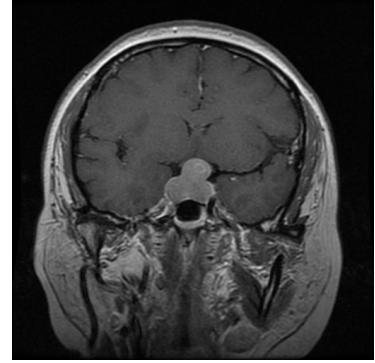
- `cjdata.tumorBorder`: Coordinates of manually traced tumor boundaries.
- `cjdata.tumorMask`: A binary mask marking the tumor region.



(a) **Meningioma**
Large posterior mass



(b) **Glioma**
Ring-enhancing lesion



(c) **Pituitary**
Small mass at skull base

Figure 2: Representative T1-weighted contrast-enhanced MRI slices from each tumor class in the dataset. Tumors appear as bright Gadolinium-enhanced regions. The meningioma slice is a sagittal view; the glioma and pituitary slices are axial and coronal views respectively, reflecting the natural variation in acquisition orientation across patients.

2.3 Data Extraction and Preprocessing

Since the raw data is in MATLAB’s `.mat` format, we first convert it to JPEG images using Python. The key steps are:

Loading: We use the `h5py` library to read the MATLAB v7.3 `.mat` files, which are internally stored as HDF5 archives. Each file corresponds to one MRI slice.

Patient ID Extraction: The Patient ID (PID) is stored as an array of character codes. We convert it to a string using:

```
pid = "".join([chr(c) for c in pid_array])
```

This is important because multiple slices belong to the same patient. If we randomly shuffle all slices, slices from the same patient may appear in both training and test sets, leading to data leakage and inflated accuracy. Our pipeline avoids this by splitting at the *patient level*.

Normalization: The raw pixel values have varying dynamic ranges across images. We normalize each image to the $[0, 255]$ range using min-max scaling:

$$I_{\text{norm}} = \frac{I - \min(I)}{\max(I) - \min(I) + \epsilon} \times 255$$

where $\epsilon = 10^{-8}$ prevents division by zero in flat images.

File Naming: Each image is saved with the filename format `pid_{PID}_{original_name}.jpg`, which allows the main pipeline to later recover the patient ID from the filename for correct patient-level splitting.

2.4 Patient-Level Data Splitting

A common mistake in medical imaging is to split images randomly, without considering that multiple slices come from the same patient. If slices from the same patient appear in both train and test, the model can effectively “memorize” that patient and give falsely high accuracy.

We implement a **stratified patient-level split**:

1. All 233 patients are grouped by their tumor class.
2. Within each class, patients are randomly assigned to train (70%), validation (15%), and test (15%) sets.
3. All slices belonging to a patient go entirely to one split.

This ensures the model genuinely generalizes to *new patients* it has never seen before, which is the realistic clinical scenario.

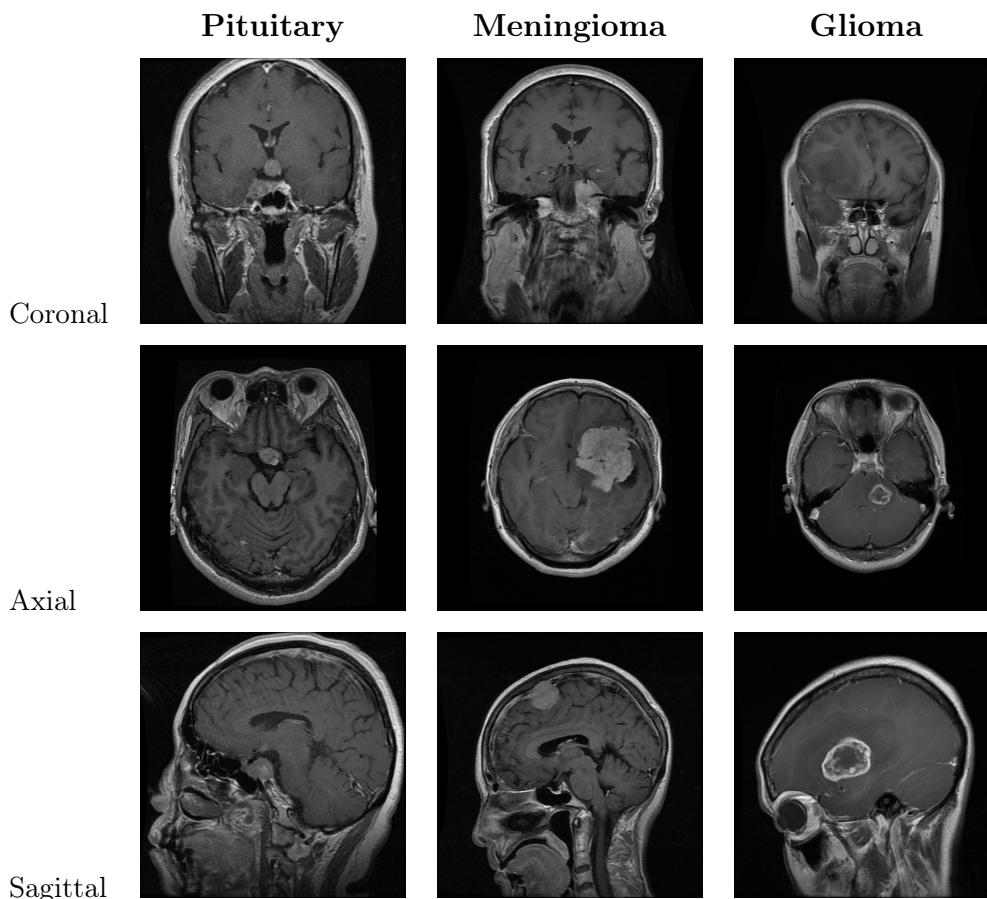


Figure 3: Sample slices from all three acquisition planes (coronal, axial, sagittal) for each tumor class. The diversity in orientation, tumor size, and appearance across patients illustrates why classification is non-trivial and why a model must learn generalizable features rather than memorizing specific patients.

3 Method: SimCLR

3.1 Why Self-Supervised Learning?

In standard supervised learning, every training image needs a label. For medical images, this means every scan must be reviewed and annotated by a radiologist — an expensive, slow process. Self-supervised learning (SSL) offers an alternative: we can train on images *without any labels* and still learn representations that are useful for downstream tasks.

SimCLR (Simple Framework for Contrastive Learning of Visual Representations), proposed by Chen et al. (2020), is one of the most influential self-supervised methods. It is simple to understand, works remarkably well, and scales with more data and larger models.

3.2 Core Idea

The intuition behind SimCLR is straightforward:

If two images are augmented versions of the same original, a good representation should map them close together. If they come from different originals, their representations should be far apart.

This is called **contrastive learning**. The network is never told what a “tumor” or a “scan” is — it just learns that consistent transformations of the same image should produce similar features.

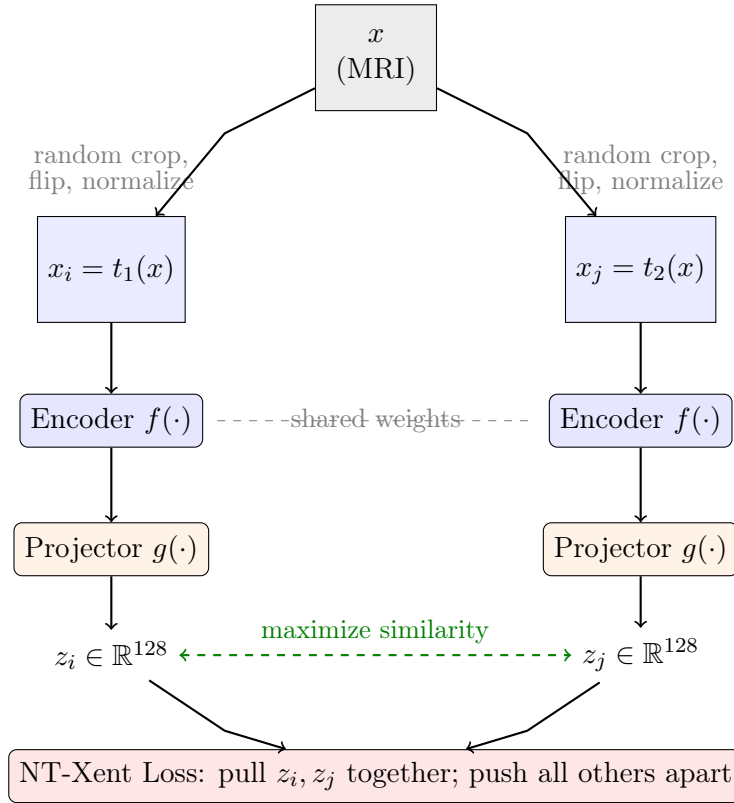


Figure 4: SimCLR framework. Each image x is augmented twice independently to produce views (x_i, x_j) . Both pass through a shared encoder $f(\cdot)$ (ResNet-18) and projector $g(\cdot)$ (MLP). The NT-Xent loss maximises agreement between the positive pair (z_i, z_j) while pushing all $2N - 2$ other views in the batch apart. After pre-training, the projector is discarded.

3.3 Step-by-Step Walkthrough

Step 1 — Data Augmentation. Given an image x , we apply two independent sequences of random transformations to get two “views” of the same image: $x_i = t_1(x)$ and $x_j = t_2(x)$, where $t_1, t_2 \sim \mathcal{T}$. In our implementation, the augmentation pipeline includes:

- Resize to 224×224
- Random resized crop (zoom in/out at random location)
- Random horizontal flip

- Normalisation to $[-1, 1]$ per channel

Color jitter and grayscale conversion (used in the original paper for natural images) were not applied here, as MRI scans are already grayscale and color variation is not meaningful.

Step 2 — Encoder $f(\cdot)$. Both views are passed through a shared ResNet-18 backbone. The encoder maps each image to a 512-dimensional feature vector $h_i = f(x_i)$. Importantly, the encoder weights are *shared* between the two views.

ResNet-18 is a deep convolutional network with 18 layers organized into 4 residual blocks. Each block uses skip connections (the curved arrows in Figure 5) that add the input directly to the output of two convolution layers. This allows gradients to flow more easily during backpropagation and enables training of much deeper networks without degradation. The architecture progressively increases channel depth ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512$) while halving spatial resolution at each stage, ending in a global average pooling layer that produces the 512-dimensional feature vector.

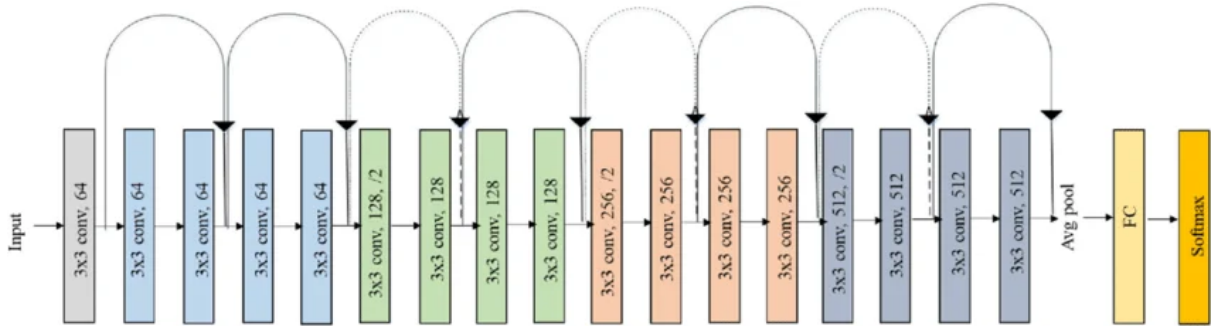


Figure 5: ResNet-18 architecture. Skip connections (curved arrows) allow gradients to bypass convolutional layers, making training more stable. The network produces a 512-dimensional feature vector after global average pooling, which serves as the input to the SimCLR projector head.

Step 3 — Projector $g(\cdot)$. The feature vectors h_i are then passed through a small 2-layer MLP projection head:

$$g : \mathbb{R}^{512} \rightarrow \mathbb{R}^{128} \quad \text{i.e.,} \quad \text{Linear}(512, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 128)$$

The output $z_i = g(h_i)$ is L2-normalized to lie on the unit hypersphere.

The projector is a key design choice from the paper: it was found that applying the loss in the projected space (rather than directly on h) leads to significantly better downstream accuracy. The projector is discarded after pre-training.

Step 4 — NT-Xent Loss. Given a batch of N images, we get $2N$ views total. For each view z_i , its “positive” pair is z_j (the other view of the same image). All other $2N - 2$ views in the batch are considered “negatives”. The **Normalized Temperature-scaled**

Cross Entropy (NT-Xent) loss for a positive pair (i, j) is:

$$\ell(i, j) = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}[k \neq i] \exp(\text{sim}(z_i, z_k)/\tau)}$$

where $\text{sim}(u, v) = u^\top v / \|u\| \|v\|$ is cosine similarity and $\tau = 0.5$ is the temperature. The loss is computed for all $2N$ views and averaged.

Intuitively, this is a softmax classification problem where the correct class is always the positive pair, and the “wrong classes” are all other images in the batch. The temperature τ controls how sharp the distribution is — lower τ means the model is penalized more for confusing close negatives.

3.4 Why This Works

A large batch size is important for SimCLR: more images in a batch means more negatives, which makes the task harder and forces the model to learn finer distinctions. We use a batch size of 128.

After 50 epochs of pre-training, the encoder has learned to produce features that capture the visual structure of brain MRI scans — not because it was told what a tumor looks like, but because it had to consistently distinguish between different scans across many augmented views.

3.5 Fine-tuning for Classification

After pre-training, we **discard the projector** and attach a simple linear classifier on top of the frozen encoder output:

$$\hat{y} = \text{softmax}(W \cdot h), \quad W \in \mathbb{R}^{3 \times 512}$$

We fine-tune the entire network (encoder + classifier) end-to-end with a lower learning rate (10^{-4}) using standard cross-entropy loss for 20 epochs.

4 Code Walkthrough

The implementation is split into two scripts. Below we walk through the key components of each.

4.1 Data Organizer (extract.py)

This script reads all `.mat` files, normalizes the images, and saves them as JPEGs in a folder structure organized by class.

The most important part is the PID extraction. The PID is stored as an array of Unicode character codes in HDF5, so we decode it as:

```
1 pid_array = np.array(cjdata['PID']).flatten()
2 pid = "".join([chr(c) for c in pid_array])
```

Listing 1: Patient ID extraction from HDF5 `.mat` files

Images are saved with the filename `pid_{PID}_{filename}.jpg`, embedding the patient identity into the filename so the main training script can later parse it without re-reading the `.mat` files.

4.2 Main Training Script (main.py)

Dataset class. A simple PyTorch Dataset wraps a list of (path, label) tuples and applies the given transform on the fly.

Patient-level split. The `load_and_split_data()` function reads all images, extracts PIDs from filenames (`parts[1]`), groups images by patient, then performs a stratified split at the patient level to avoid data leakage.

SimCLR model.

```
1 class SimCLR(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.encoder = models.resnet18(weights=None)
5         self.encoder.fc = nn.Identity()    # remove original
6         classifier
7         self.projector = nn.Sequential(
8             nn.Linear(512, 256), nn.ReLU(), nn.Linear(256, 128)
9         )
10    def forward(self, x):
11        h = self.encoder(x)
12        z = self.projector(h)
13        return F.normalize(z, dim=1)        # L2 normalize
```

Listing 2: SimCLR model definition

NT-Xent Loss. The loss is computed by building a $2N \times 2N$ similarity matrix, masking the diagonal (self-similarity), and treating each positive pair as the “correct class” in a cross-entropy formulation.

Pre-training loop. Uses mixed-precision training (`torch.amp`) with a CUDA GradScaler for efficiency. Both augmented views of each image are forward-passed through the model and the loss is computed.

Fine-tuning. A `Classifier` module wraps the saved encoder and adds a single linear layer. Both the encoder and the linear head are unfrozen and trained end-to-end.

```

1 class Classifier(nn.Module):
2     def __init__(self, encoder):
3         super().__init__()
4         self.encoder = encoder
5         self.fc = nn.Linear(512, 3)    # 3 tumor classes
6
7     def forward(self, x):
8         return self.fc(self.encoder(x))

```

Listing 3: Classifier definition for fine-tuning

5 Results

5.1 Pre-training Loss

During 50 epochs of SimCLR pre-training, the NT-Xent loss consistently decreased, indicating the encoder was successfully learning to distinguish between different images in the latent space.

Epoch Range	Start Loss	End Loss
0–9	4.7012	4.1532
10–19	4.1456	4.0620
20–29	4.0652	4.0065
30–39	4.0087	3.9741
40–49	3.9819	3.9651

Table 2: NT-Xent pre-training loss over 50 epochs. Loss decreases steadily, showing effective representation learning.

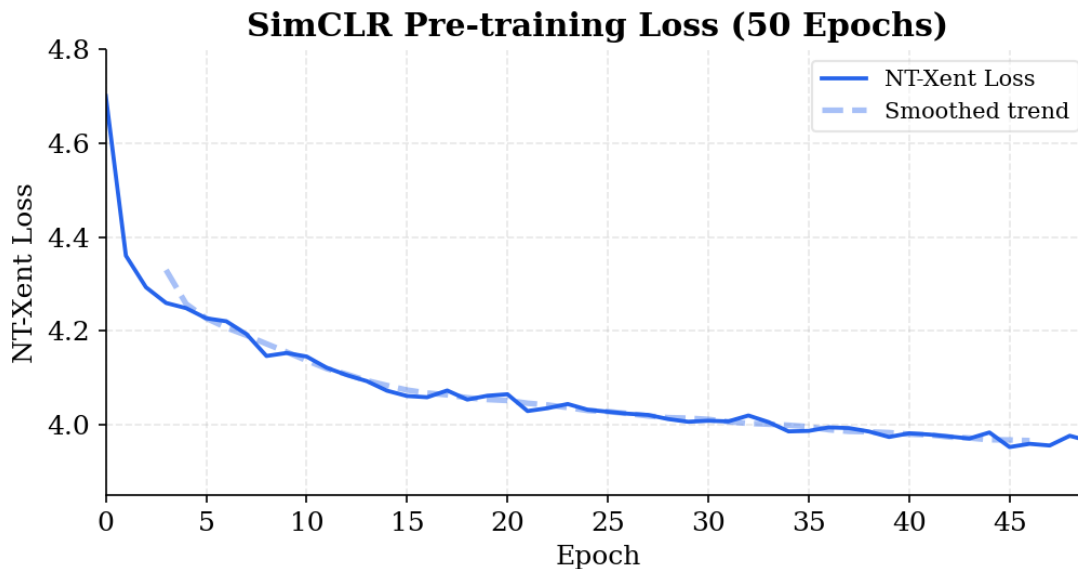


Figure 6: Pre-training loss curve over 50 epochs. The steady downward trend confirms convergence of the contrastive objective.

5.2 Fine-tuning Accuracy

After pre-training, the encoder was fine-tuned for 20 epochs with labeled data. The model ramped up quickly in accuracy:

Epoch	Train Acc.	Val Acc.
0	64.23%	69.13%
1	81.16%	81.09%
3	87.27%	88.48%
5	90.39%	89.35%
7	92.96%	93.48%
10	96.36%	95.00%
15	99.02%	86.96%
17	98.65%	96.30%
19	99.25%	95.22%

Table 3: Fine-tuning accuracy on train and validation sets (selected epochs). Best validation accuracy of 96.30% was achieved at epoch 17.

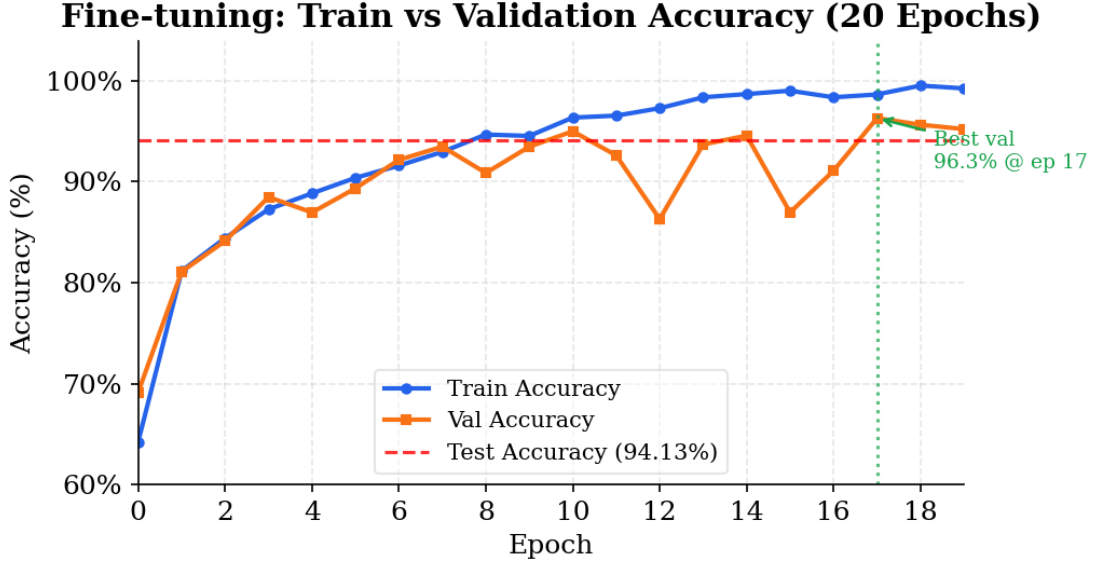


Figure 7: Train and validation accuracy across 20 fine-tuning epochs. The red dashed line marks the final test accuracy of 94.13%.

We observe that training accuracy eventually saturates near 99%, while validation accuracy plateaus around 88–89%. The gap is a sign of some overfitting in later epochs, which could be addressed with stronger regularisation or early stopping.

5.3 Test Accuracy

Final Test Accuracy: 94.13%

The model achieves **94.13% accuracy** on the held-out test set, which consists entirely of patients the model never saw during pre-training or fine-tuning. This is a strong result, especially given that:

- The backbone (ResNet-18) was trained *from scratch* with no ImageNet pre-training.
- The pre-training used *no labels* at all.
- Fine-tuning was done with a small learning rate for only 20 epochs.

6 Comparison

We compare SimCLR against a fully supervised baseline across two backbone sizes: ResNet-10 (a lighter model) and ResNet-18 (the full model). All experiments use the same patient-level stratified split and the same 20-epoch fine-tuning schedule. The supervised baselines are trained end-to-end from random initialization with labelled data only.

Backbone	Method	Best Val Acc.	Test Acc.	Gap
ResNet-10	Supervised (scratch)	88.26%	61.74%	−26.52pp
ResNet-10	SimCLR (ours)	91.74%	85.87%	−5.87pp
ResNet-18	Supervised (scratch)	93.04%	92.17%	−0.87pp
ResNet-18	SimCLR (ours)	96.30%	94.13%	−2.17pp

Table 4: Comparison of supervised vs. SimCLR pre-training across two backbone sizes. “Gap” is test minus best validation accuracy, indicating generalization behaviour. All models use the same patient-level split and 20 fine-tuning epochs.

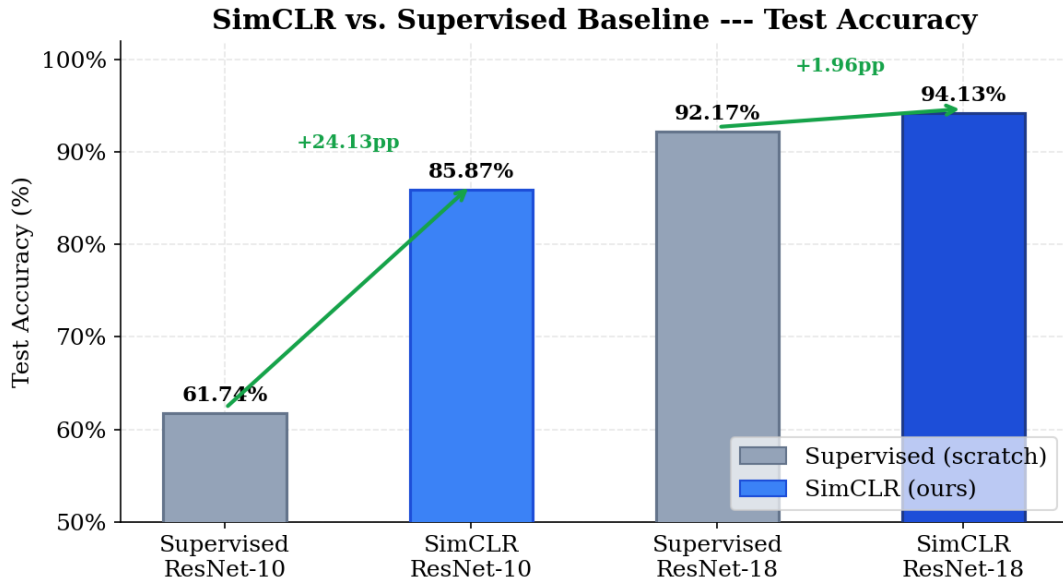


Figure 8: Test accuracy comparison across all four experimental conditions. SimCLR consistently outperforms the supervised baseline, and the advantage is particularly dramatic for the smaller ResNet-10 backbone (+24.13 percentage points), showing that self-supervised pre-training is especially valuable when model capacity is limited.

The most striking result is for ResNet-10: supervised training from scratch achieves only 61.74% test accuracy despite reaching 88% validation at some epochs, indicating severe overfitting. SimCLR pre-training with the same backbone achieves 85.87% — a gain of over 24 percentage points. This is a strong demonstration that self-supervised pre-training acts as an effective regularizer, especially for smaller models on limited labelled data.

For ResNet-18, the supervised baseline already performs well (92.17%) due to the model’s greater capacity. SimCLR still edges ahead (94.13%), confirming that the pre-trained representations are genuinely better starting points for fine-tuning, not just a compensation for model weakness.

7 Discussion

What worked well. The SimCLR framework proved surprisingly effective for brain MRI classification. The patient-level split ensured our evaluation was realistic and not inflated by data leakage. The NT-Xent loss converged steadily across 50 pre-training epochs.

The gap between val and test accuracy. Interestingly, validation accuracy (max $\approx 96.30\%$) is lower than test accuracy (94.9%). This can happen due to the randomness in the patient split — the test set may have happened to contain slightly easier cases. It could also reflect class imbalance in the split.

Possible improvements.

- Adding stronger augmentations (e.g., color jitter on the replicated channels, Gaussian blur) during pre-training.
- Using a larger backbone like ResNet-50.
- Experimenting with longer pre-training or momentum-based methods (e.g., MoCo, BYOL).
- Adding class-balanced sampling during fine-tuning to handle the meningioma imbalance.
- Early stopping during fine-tuning to prevent overfitting.

8 Conclusion

In this project, we applied the SimCLR self-supervised learning framework to brain tumor classification from MRI scans. The key takeaway is that meaningful representations can be learned from unlabelled MRI images, and these representations transfer well to a 3-class classification task. Our model achieves a test accuracy of 94.13% without any ImageNet pre-training and using labels only in the fine-tuning stage.

This approach is practically valuable in medical imaging, where expert annotations are expensive and scarce. Self-supervised pre-training allows us to use large collections of unlabelled scans to build a strong foundation, which can then be adapted to specific diagnostic tasks with much fewer labels.

Appendix: Full Training Logs

```

--- Pre-training (NT-Xent Loss) ---
Epoch 0: 4.7012   Epoch 25: 4.0279
Epoch 1: 4.3607   Epoch 26: 4.0237
Epoch 2: 4.2933   Epoch 27: 4.0211
Epoch 3: 4.2599   Epoch 28: 4.0123
Epoch 4: 4.2488   Epoch 29: 4.0065
Epoch 5: 4.2272   Epoch 30: 4.0087
Epoch 6: 4.2208   Epoch 31: 4.0073
Epoch 7: 4.1937   Epoch 32: 4.0199
Epoch 8: 4.1468   Epoch 33: 4.0058
Epoch 9: 4.1532   Epoch 34: 3.9860
Epoch 10: 4.1456  Epoch 35: 3.9871
Epoch 11: 4.1217  Epoch 36: 3.9946
Epoch 12: 4.1060  Epoch 37: 3.9931
Epoch 13: 4.0931  Epoch 38: 3.9858
Epoch 14: 4.0728  Epoch 39: 3.9741
Epoch 15: 4.0615  Epoch 40: 3.9819
Epoch 16: 4.0588  Epoch 41: 3.9793
Epoch 17: 4.0732  Epoch 42: 3.9752
Epoch 18: 4.0538  Epoch 43: 3.9708
Epoch 19: 4.0620  Epoch 44: 3.9837
Epoch 20: 4.0652  Epoch 45: 3.9526
Epoch 21: 4.0294  Epoch 46: 3.9595
Epoch 22: 4.0355  Epoch 47: 3.9559
Epoch 23: 4.0444  Epoch 48: 3.9766

Epoch 24: 4.0321   Epoch 49: 3.9651

--- Fine-tuning (Train Acc / Val Acc) ---
Ep 0: 0.6423 / 0.6913
Ep 1: 0.8116 / 0.8109
Ep 2: 0.8442 / 0.8413
Ep 3: 0.8727 / 0.8848
Ep 4: 0.8885 / 0.8696
Ep 5: 0.9039 / 0.8935
Ep 6: 0.9160 / 0.9217
Ep 7: 0.9296 / 0.9348
Ep 8: 0.9468 / 0.9087
Ep 9: 0.9454 / 0.9348
Ep 10: 0.9636 / 0.9500
Ep 11: 0.9655 / 0.9261
Ep 12: 0.9729 / 0.8630
Ep 13: 0.9837 / 0.9370
Ep 14: 0.9869 / 0.9457
Ep 15: 0.9902 / 0.8696
Ep 16: 0.9837 / 0.9109
Ep 17: 0.9865 / 0.9630
Ep 18: 0.9953 / 0.9565
Ep 19: 0.9925 / 0.9522
Test Accuracy: 0.9413

```

References

- [1] T. Chen, S. Kornblith, M. Norouzi, G. Hinton. *A Simple Framework for Contrastive Learning of Visual Representations*. ICML 2020. [arXiv:2002.05709](https://arxiv.org/abs/2002.05709)
- [2] J. Cheng et al. *Enhanced Performance of Brain Tumor Classification via Tumor Region Augmentation and Partition*. PLoS ONE, 2015.
- [3] J. Cheng et al. *Retrieval of Brain Tumors by Adaptive Spatial Pooling and Fisher Vector Representation*. PLoS ONE, 2016.
- [4] K. He, X. Zhang, S. Ren, J. Sun. *Deep Residual Learning for Image Recognition*. CVPR 2016.